

ОТЧЁТ

по итоговому домашнему заданию

«Детекция и подсчёт автомобилей на видео с использованием YOLOv8»

Студент:	Морозова Юлия Андреевна
Преподаватель:	Ярослав Найчук
Дисциплина:	Компьютерное зрение
Дата:	23.05.2026

1. Введение

Данное итоговое домашнее задание посвящено разработке системы автоматического подсчёта автомобилей, проезжающих через заданную линию на видеозаписи. В рамках работы реализованы детекция транспортных средств с использованием нейросетевой модели YOLOv8, трекинг объектов между кадрами, логика подсчёта пересечений контрольной линии и визуализация результатов в реальном времени.

Практическая значимость подобных систем очень высока: автоматический подсчёт транспортных потоков применяется в городском планировании, управлении дорожным движением, мониторинге загруженности дорог и работе «умного города».

2. Описание задания

Задание включало пять основных этапов и два дополнительных:

- Этап 0 — поиск подходящего видео с автомобилями.
- Этап 1 — детекция автомобилей YOLOv8 + трекинг с уникальными ID.
- Этап 2 — задание линии подсчёта по двум точкам.
- Этап 3 — логика подсчёта: одно направление, без повторного счёта.
- Этап 4 — визуализация: рамки, ID, линия, счётчик.
- Этап 5 — тестирование на реальном видео.

3. Анализ выполнения задания

В таблице ниже приведена сводная оценка выполнения каждого этапа:

Этап / Задание	Статус	Примечание
Этап 0 — поиск видео	Выполнено	Видео 12345.mp4
Этап 1 — детекция YOLOv8 (класс car)	Выполнено	Модель yolov8n.pt
Этап 1 — трекинг (model.track())	Выполнено	Встроенный ByteTrack (Ultralytics)
Этап 1 — отображение bbox + ID	Выполнено	results[0].plot()
Этап 2 — линия по двум точкам	Выполнено	Переменная line_coords
Этап 2 — отображение линии на кадре	Выполнено	cv2.line()
Этап 3 — центр bbox как точка трека	Выполнено	cx, cy вычислены
Этап 3 — проверка пересечения линии	Выполнено	line_intersection()
Этап 3 — подсчёт только одного направления	Выполнено	left_to_right / top_to_bottom

Этап / Задание	Статус	Примечание
Этап 3 — исключение повторного подсчёта	Выполнено	counted_ids (set)
Этап 4 — bbox, ID, центр объекта	Выполнено	Кружок + текст ID
Этап 4 — счётчик в углу кадра	Выполнено	cv2.putText()
Этап 5 — тестирование на видео	Выполнено	Google Colab notebook

4. Подробное описание реализации

4.1 Этап 0 — Подготовка видео

Для тестирования использовалось видео с городской камеры наблюдения (файл 12345.mp4), на котором хорошо различимы автомобили, движущиеся сверху вниз через кадр. Именно это определило выбор направления `top_to_bottom` при запуске.

4.2 Этап 1 — Детекция и трекинг

Для детекции применялась модель YOLOv8n (nano) из библиотеки Ultralytics. Отображаются только транспортные средства: классы 2 (car), 3 (motorcycle), 5 (bus) и 7 (truck) датасета COCO. Фильтрация выполняется через маску:

```
vehicle_classes = [2, 3, 5, 7]
mask = np.isin(boxes.cls.cpu().numpy(), vehicle_classes)
```

Для трекинга используется встроенный механизм `model.track()` из библиотеки Ultralytics (алгоритм ByteTrack). Он автоматически назначает каждому объекту уникальный ID и сохраняет его между кадрами без необходимости писать собственный трекер. Параметры запуска трекинга:

- `conf = 0.5` — порог уверенности детекции.
- `iou = 0.5` — порог IoU для подавления дублирующих детекций (NMS).

4.3 Этап 2 — Линия подсчёта

Линия задаётся через переменную `line_coords` в формате `x1,y1,x2,y2` прямо в ячейке Google Colab. Поддерживаются два режима:

- `left_to_right` — вертикальная линия, растянутая по всей высоте кадра.
- `top_to_bottom` — горизонтальная линия, растянутая по всей ширине кадра.

Линия автоматически масштабируется на весь кадр, поэтому достаточно указать лишь одну координату (X для вертикальной, Y для горизонтальной):

```
# top_to_bottom: линия по Y-координате
line_y = pt1_in[1]
```

```
line_pt1, line_pt2 = (0, line_y), (w, line_y)
```

Пример настройки параметров для режима `top_to_bottom` с линией `Y = 800`:

```
direction = 'top_to_bottom'
```

4.4 Этап 3 — Логика подсчёта пересечений

Центроид каждого трекируемого объекта вычисляется как середина bounding box. На каждом кадре для каждого объекта проверяется, пересекает ли отрезок (предыдущий_центроид → текущий_центроид) заданную линию. Для проверки используется классический алгоритм на основе знака ориентации треугольников (CCW):

```
def line_intersection(p1, p2, p3, p4):
    def ccw(a, b, c):
        return (c[1]-a[1])*(b[0]-a[0]) > (b[1]-a[1])*(c[0]-a[0])
    return ccw(p1,p3,p4) != ccw(p2,p3,p4) and ccw(p1,p2,p3) != ccw(p1,p2,p4)
```

После подтверждения пересечения проверяется направление движения:

```
if direction == 'top_to_bottom' and prev[1] < line_y <= cy:
    cross_ok = True
```

Уникальный ID объекта добавляется в множество `counted_ids`, что гарантирует однократный подсчёт каждого автомобиля:

```
if cross_ok and obj_id not in counted_ids:
    total_crossings += 1
    counted_ids.add(obj_id)
```

4.5 Этап 4 — Визуализация

На каждый кадр наносятся следующие элементы:

1. Bounding boxes с классом и ID объекта (отрисовка через `cv2.rectangle` и `cv2.putText` — только для транспортных средств).
2. Идентификатор трека — текст «ID:N» над bounding box.
3. Текущий центроид — жёлтая точка (`cv2.circle`) в центре объекта.
4. Линия подсчёта — красная линия (`cv2.line`, BGR 0,0,255).
5. Счётчик пересечений — текст «Crossings: N» в левом верхнем углу.

4.6 Этап 5 — Запуск и тестирование

Notebook реализован в Google Colab. Зависимости устанавливаются в первой ячейке:

```
!pip install ultralytics opencv-python numpy lap --quiet
direction = 'top_to_bottom' # left_to_right или top_to_bottom
line_coords = "0,800,0,800" # для top_to_bottom: 0,Y,0,Y
```

```
python MorozovaKZ.py --video 12345.mp4 --direction top_to_bottom --line  
0,1000,0,1000
```

Параметры задаются в ячейке «Настройка параметров», после чего запускается ячейка обработки. Результатом является видеофайл output_video.mp4, который можно скачать из Colab. В процессе выводится прогресс:

```
Обработано кадров: 100, пересечений: 3
```

```
☒ Готово! Всего пересечений: N
```

5. Заключение

В ходе выполнения итогового домашнего задания была разработана работающая система подсчёта транспортных средств на основе детектора YOLOv8 в среде Google Colab. Все этапы задания (0–5) выполнены в полном объёме: реализованы детекция с фильтрацией по классам ТС, трекинг через встроенный model.track() (ByteTrack), логика подсчёта пересечений и визуализация. Notebook успешно запускается на реальном видео.

Разработанное решение демонстрирует понимание всей цепочки: установка зависимостей → детекция → трекинг → логика счёта → визуализация.